

**(20 pts)**

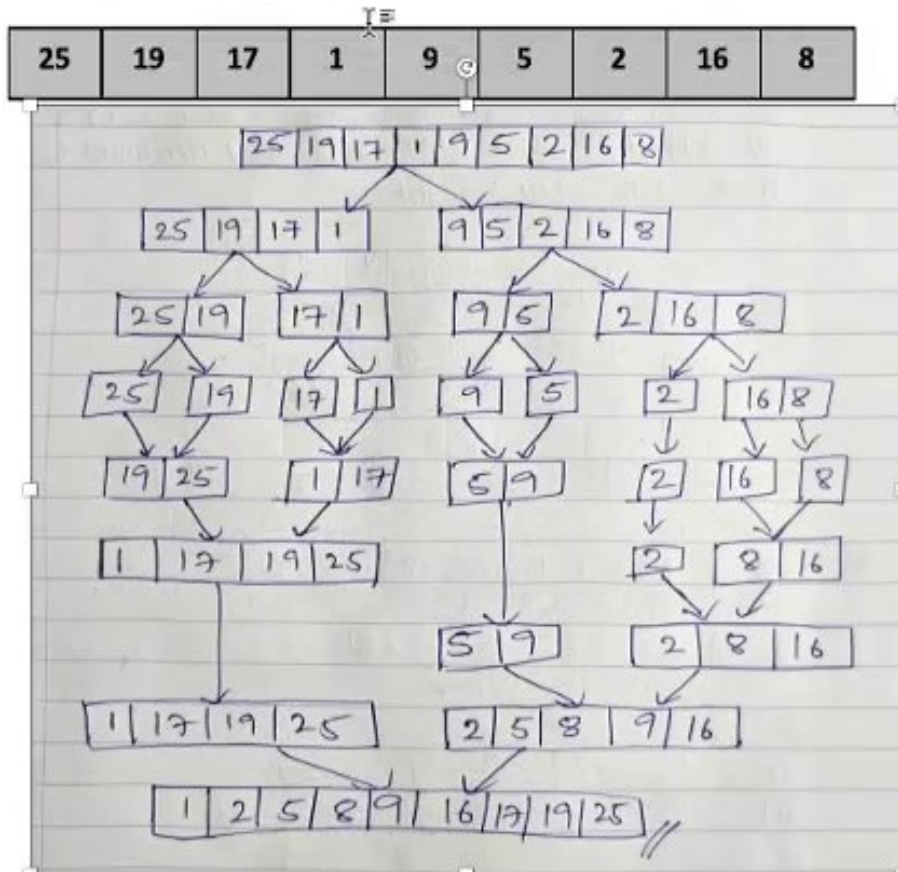
1. Show the diagram (recursion tree) of divide-and-conquer MergeSort algorithm on the numbers below.

|    |    |    |   |   |   |   |    |   |
|----|----|----|---|---|---|---|----|---|
| 25 | 19 | 17 | 1 | 9 | 5 | 2 | 16 | 8 |
|----|----|----|---|---|---|---|----|---|

Solution — Recursion Tree:

**(20 pts)**

1. Show the diagram (recursion tree) of divide-and-conquer MergeSort algorithm on the numbers below. If you implement the algorithm,

**NOTE:** Implementation is extra 20 points.

(60 pts)

2. Find the MST and its weight using Prim's, Kruskal's and Reverse Delete algorithms.

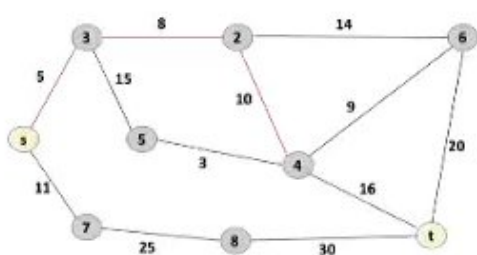
Graph edges:  $s-3(5)$ ,  $s-7(11)$ ,  $3-2(8)$ ,  $3-5(15)$ ,  $2-6(14)$ ,  $2-4(10)$ ,  $5-4(3)$ ,  $4-6(9)$ ,  $4-t(16)$ ,  $6-t(20)$ ,  $7-8(25)$ ,  $8-t(30)$ **PRIM'S ALGORITHM (start from vertex s)**

At each step pick the minimum-weight edge crossing from the visited set to an unvisited vertex.

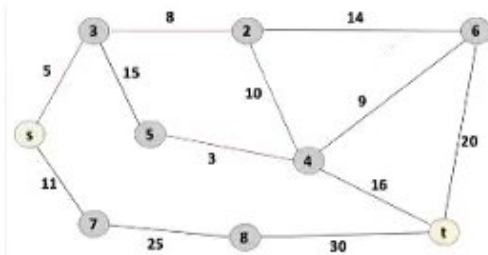
| Step | Edge  | Wt | Visited Set                 | Action | Note                                        |
|------|-------|----|-----------------------------|--------|---------------------------------------------|
| —    | —     | —  | {s}                         | —      | Begin at s                                  |
| 1    | s - 3 | 5  | {s, 3}                      | ADD ?  | min edge from s: s-3(5) < s-7(11)           |
| 2    | 3 - 2 | 8  | {s, 3, 2}                   | ADD ?  | min crossing: 3-2(8)                        |
| 3    | 2 - 4 | 10 | {s, 3, 2, 4}                | ADD ?  | min crossing: 2-4(10) < 3-5(15)             |
| 4    | 4 - 5 | 3  | {s, 3, 2, 4, 5}             | ADD ?  | min crossing: 4-5(3)                        |
| 5    | 4 - 6 | 9  | {s, 3, 2, 4, 5, 6}          | ADD ?  | min crossing: 4-6(9) < 2-6(14)              |
| 6    | s - 7 | 11 | {s, 3, 2, 4, 5, 6, 7}       | ADD ?  | min crossing: s-7(11)                       |
| 7    | 4 - t | 16 | {s, 3, 2, 4, 5, 6, 7, t}    | ADD ?  | min crossing: 4-t(16) < 6-t(20)             |
| 8    | 7 - 8 | 25 | {s, 3, 2, 4, 5, 6, 7, t, 8} | ADD ?  | min crossing: 7-8(25). All vertices in MST! |

**Prim's MST Weight =  $5 + 8 + 10 + 3 + 9 + 11 + 16 + 25 = 87$**

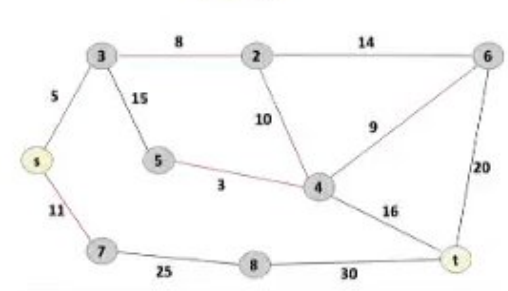
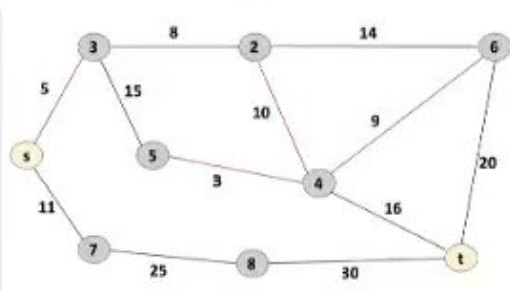
Step-by-step graph (Prim's steps 5–8):



(5)



(6)



**KRUSKAL'S ALGORITHM**

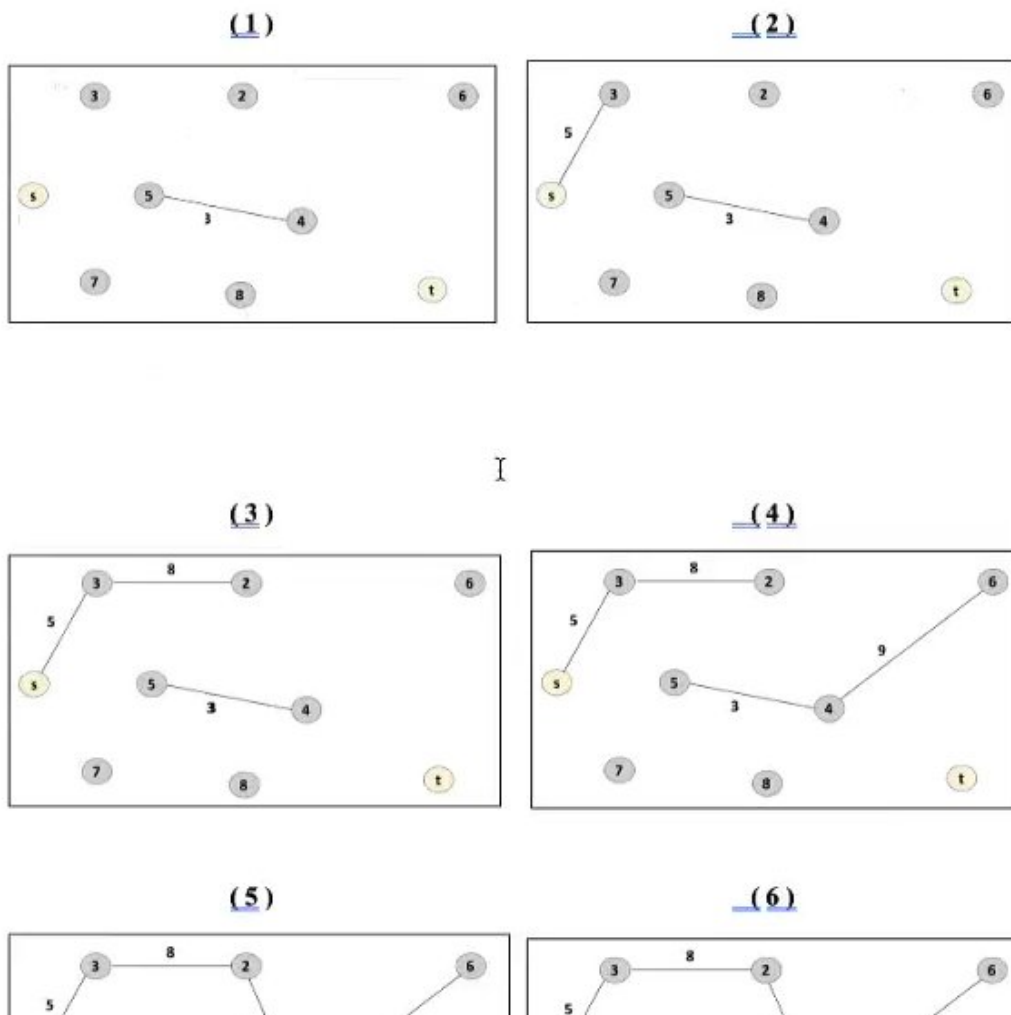
Sort ALL edges by weight (ascending). Greedily add each edge if it does NOT form a cycle (union-find).

**Sorted edges:** 4-5(3) s-3(5) 3-2(8) 4-6(9) 2-4(10) s-7(11) 2-6(14) 3-5(15) 4-t(16) 6-t(20) 7-8(25) 8-t(30)

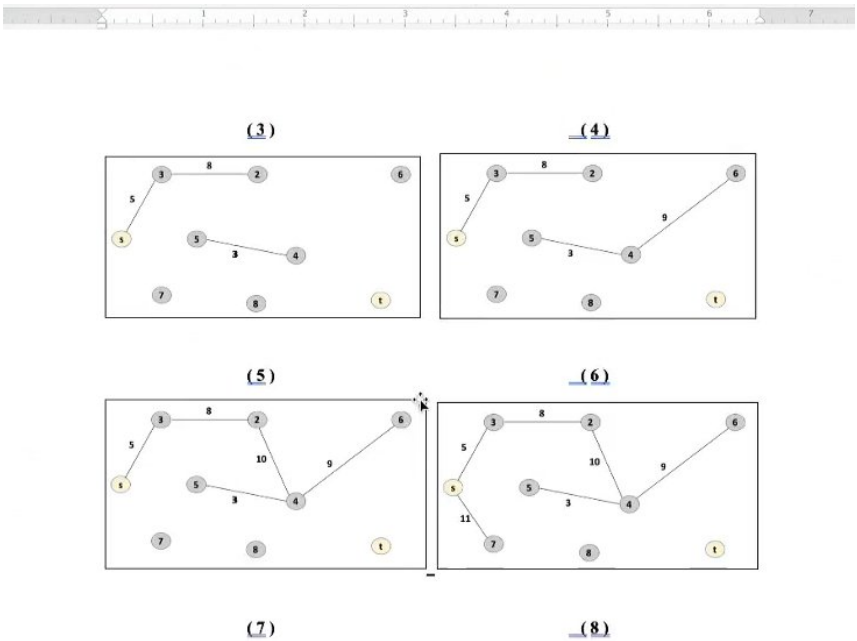
| Step | Edge | Wt | Cycle? | Action | Components                                 |
|------|------|----|--------|--------|--------------------------------------------|
| 1    | 4-5  | 3  | No     | ADD ?  | {s} {3} {2} {6} {4,5} {7} {8} {t}          |
| 2    | s-3  | 5  | No     | ADD ?  | {s,3} {2} {6} {4,5} {7} {8} {t}            |
| 3    | 3-2  | 8  | No     | ADD ?  | {s,3,2} {6} {4,5} {7} {8} {t}              |
| 4    | 4-6  | 9  | No     | ADD ?  | {s,3,2} {4,5,6} {7} {8} {t}                |
| 5    | 2-4  | 10 | No     | ADD ?  | {s,3,2,4,5,6} {7} {8} {t}                  |
| 6    | s-7  | 11 | No     | ADD ?  | {s,3,2,4,5,6,7} {8} {t}                    |
| 7    | 2-6  | 14 | YES    | SKIP ? | 2 and 6 already in same component          |
| 8    | 3-5  | 15 | YES    | SKIP ? | 3 and 5 already in same component          |
| 9    | 4-t  | 16 | No     | ADD ?  | {s,3,2,4,5,6,7,t} {8}                      |
| 10   | 6-t  | 20 | YES    | SKIP ? | 6 and t already connected                  |
| 11   | 7-8  | 25 | No     | ADD ?  | {s,3,2,4,5,6,7,t,8} — All 9 vertices done! |
| 12   | 8-t  | 30 | YES    | SKIP ? | All vertices already in one component      |

**Kruskal's MST Weight = 3 + 5 + 8 + 9 + 10 + 11 + 16 + 25 = 87**

Step-by-step graph diagrams (Kruskal's steps 1–4):

**KRUSKAL'S ALGORITHM**

Kruskal's step-by-step graph (steps 3–8):



REVERSE DELETE ALGORITHM

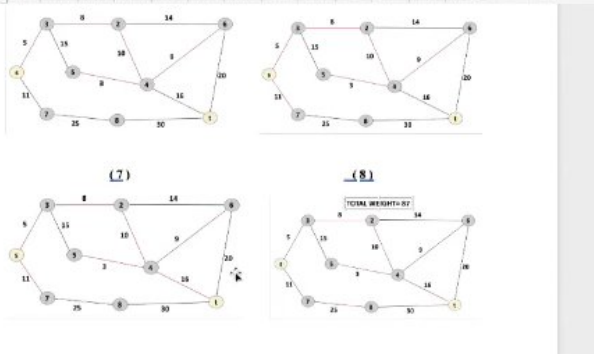
Start with the FULL graph. Consider edges heaviest-first. Remove an edge ONLY if graph stays connected.

| Step | Edge | Wt | Decision | Reason                                            |
|------|------|----|----------|---------------------------------------------------|
| 1    | 8-t  | 30 | Remove ✓ | Graph still connected — 8 reachable via 7-8 → s-7 |
| 2    | 7-8  | 25 | KEEP ✗   | Removing isolates vertex 8 (only edge left to 8)  |
| 3    | 6-t  | 20 | Remove ✓ | t still connected via 4-t; 6 still via 4-6        |
| 4    | 4-t  | 16 | KEEP ✗   | Removing disconnects t (8-t already gone)         |
| 5    | 3-5  | 15 | Remove ✓ | 5 reachable via 4-5 and 2-4 and 3-2 and s-3       |
| 6    | 2-6  | 14 | Remove ✓ | 6 still reachable via 4-6                         |
| 7    | s-7  | 11 | KEEP ✗   | Removing disconnects 7 and 8 from graph           |
| 8    | 2-4  | 10 | KEEP ✗   | Removing disconnects {s,3,2} from {4,5,6,7,t,8}   |
| 9    | 4-6  | 9  | KEEP ✗   | Removing disconnects 6 (2-6 already removed)      |
| 10   | 3-2  | 8  | KEEP ✗   | Removing disconnects 2 side from s-3              |
| 11   | s-3  | 5  | KEEP ✗   | Removing disconnects s from all others            |
| 12   | 4-5  | 3  | KEEP ✗   | Removing disconnects vertex 5                     |

Reverse Delete MST Weight = 25+16+11+10+9+8+5+3 = 87

All 3 Algorithms produce the same MST — Total Weight = 87  
Edges: s-3(5), 3-2(8), 2-4(10), 4-5(3), 4-6(9), s-7(11), 4-t(16), 7-8(25)

Final MST graph (panel 8, total weight = 87).

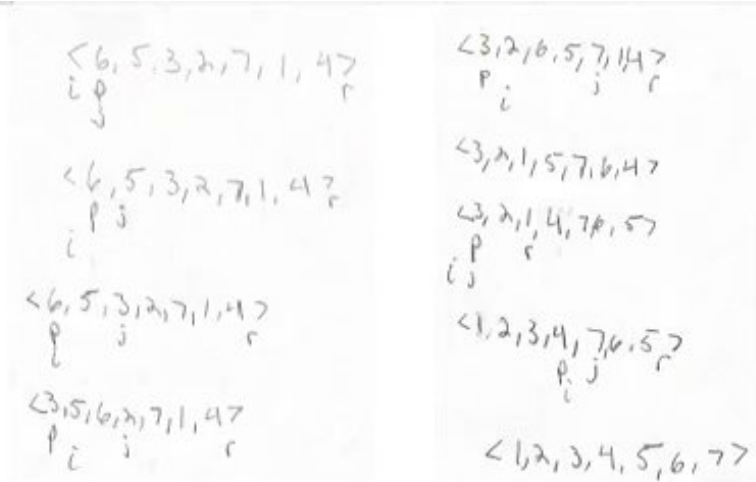


**(20 points)**

3. Sort  $S = \{6, 5, 3, 2, 7, 1, 4\}$  using divide-and-conquer Quicksort.

Show the steps (tracers) of partitioning using pointers.

*Solution (pointer trace — handwritten):*

**TURNIN:**

Save your assignment as one pdf file, name it using "lastname-firstname" format and submit to the Pilot by the deadline

**PARTITION trace — PARTITION(A, p, r), pivot = A[r]**

| Action / Pointer state                           | Operation                                                      | Array / Result                              |
|--------------------------------------------------|----------------------------------------------------------------|---------------------------------------------|
| Initial                                          | [6, 5, 3, 2, 7, 1   4]                                         | pivot $x=4$ , $i=p-1=-1$                    |
| $j=0$ : $A[0]=6 > 4$                             | no swap                                                        | $i=-1$ , array unchanged                    |
| $j=1$ : $A[1]=5 > 4$                             | no swap                                                        | $i=-1$ , array unchanged                    |
| $j=2$ : $A[2]=3 \leq 4$                          | $i=0$ , swap $A[0] \leftrightarrow A[2]$                       | [3, 5, 6, 2, 7, 1, 4]                       |
| $j=3$ : $A[3]=2 \leq 4$                          | $i=1$ , swap $A[1] \leftrightarrow A[3]$                       | [3, 2, 6, 5, 7, 1, 4]                       |
| $j=4$ : $A[4]=7 > 4$                             | no swap                                                        | $i=1$ , array unchanged                     |
| $j=5$ : $A[5]=1 \leq 4$                          | $i=2$ , swap $A[2] \leftrightarrow A[5]$                       | [3, 2, 1, 5, 7, 6, 4]                       |
| End of loop                                      | swap $A[i+1] \leftrightarrow A[r] = A[3] \leftrightarrow A[6]$ | [3, 2, 1, 4, 7, 6, 5] ← 4 in final position |
| Recurse LEFT                                     | PARTITION([3, 2, 1], $p=1$ )                                   | pivot=1 → [1, 2, 3]; 1 at index 0           |
| L-Right sub                                      | PARTITION([2, 3])                                              | pivot=3 → [2, 3] done                       |
| Recurse RIGHT                                    | PARTITION([7, 6, 5], $p=5$ )                                   | pivot=5 → [5, 6, 7]; 5 at index 0           |
| R-Right sub                                      | PARTITION([6, 7])                                              | pivot=7 → [6, 7] done                       |
| <b>Final Sorted Array: [1, 2, 3, 4, 5, 6, 7]</b> |                                                                |                                             |